

Research Take-Home

abundant-take-home · v1.8.0 · Confidential — prepared for the addressed candidate

Background

Glossary

- **Harbor** — open-source framework we use to author tasks, run agents against them, and capture trajectories. Docs: harborframework.com/docs/getting-started.
- **Task** — a single problem with a verifiable outcome. Has an instruction, an environment, and a verifier that scores the agent's final state.
- **Environment** — the sandboxed world the agent acts in: container image, available tools, MCP servers, internet access, files on disk.
- **Agent** — the LLM-plus-scaffolding under test (e.g., Claude Code, Codex, Terminus-2). Takes the instruction, acts in the environment, produces a final state.
- **Verifier** — code that inspects the post-execution state and emits a reward (in Harbor: a `tests/test.sh` driver that writes to `/logs/verifier/reward.{txt,json}`).
- **Reward** — the score, scalar or multi-criterion. The training signal.
- **Trajectory** — the captured execution: messages, tool calls, observations, reasoning, final output. Harbor emits these as ATIF JSON automatically when you `harbor run` .
- **Headroom** — the gap between current SOTA performance on a capability and what's achievable. The thing RL data is supposed to close.

About this exercise

Designed to mirror an actual day in the life of an Abundant researcher. You'll receive a (lightly fictionalized) customer brief, identify headroom, build a small sample in Harbor format, and deliver it the way we deliver to frontier labs. Turn it around within 48 hours of receiving this brief.

The Ask

Forwarded email from a head of post-training at a frontier AI lab. You're the Abundant employee running point.

Subject: RL data for long-horizon SWE

We have headroom we can't close internally on long-horizon agentic software engineering. Send us a sample task pack in Harbor format showing where exactly models in our class are failing, what the verifiable task format would look like, and how you'd scale to thousands of tasks.

Pick a sub-domain in **long-horizon SWE**. By "long-horizon", we mean tasks that will take an agent **more than 75 steps** to accomplish—that's our working definition, but step count is only one yardstick, and we encourage you to think about other ways to measure "horizon". The narrower, the better. Your narrowing *is* part of what we're evaluating.

What we're assessing

- **Headroom-finding** — taste in identifying where exactly SOTA breaks in your chosen sub-domain, and ability to ground that claim in evidence.
- **Task crafting** — quality of the sample(s): well-scoped, robust verification, actually targets the failure mode you claim, conforms to Harbor format.
- **Scale and customer delivery** — a credible plan to go from sample to a production-scale pack, presented in a form a frontier-lab researcher would want to read.

Deliverables

A single zip with this layout:

```
samples/           # One Harbor-format task per subdirectory
jobs/<job>/<trial>/ # `harbor run` output on your sample tasks, exactly as Harbor wrote
it
archive/           # Any tasks you built that you are not including in your final task
pack, one Harbor-format task per directory
report/            # Single report, your choice of format
```

- `samples/` — **this directory should only include the tasks you're submitting to the client as the final task pack**. At least 3 fully built tasks in Harbor format (see the example task in your workspace, `restaurant-weekly-cost-control-audit/`). More are welcome!
- `jobs/` — **this directory should only include trials from your retained tasks**. Contains captured output from `harbor run -p <task-path> -a mini-swe-agent -m anthropic/claude-sonnet-5`, with Harbor's own directory structure intact: every trial sits at `jobs/<job>/<trial>/`, holding `result.json`, `agent/trajectory.json`, and `verifier/`. Gather the runs for all of your tasks under that one top-level `jobs/` — don't flatten, rename, or re-nest them. Before you zip, `ls jobs/*/*/result.json` should print one line per trial you're submitting.
- `archive/` — please place any built tasks here that you discarded as part of your research process but still believe are useful artifacts for us to review, e.g. they are referenced in your final report.

- `report/` — single report, with **HUMAN-WRITTEN** prose. If we wanted to have an agent summarize your work, we'd ask one ourselves. You choose the format: md, pdf, html, slides — whatever you'd send a customer. You're more than welcome to use agents to assist with the *presentation* of your work. Your report must do three jobs:
 - **Demonstrate samples.** Walk through your sample(s) as evidence. What capability are you targeting? How did you probe SOTA? What's the landscape for this domain today (quote external studies and benchmark results)? What failure modes did you find in your own tasks? Quote from your trajectories.
 - **Clearly state your pack's headroom, and then defend it.** Why will your task pack meaningfully improve the performance of the customer's model?
 - **Lay out the scale plan.** How do you go from N to 1,000 tasks at production quality? Name the public data sources, the augmentation/enhancement strategy, your QA loop, etc.

Keep in mind that the report is customer-facing, so make it concise and engaging! Add visual aids if needed.

This is your BEST opportunity to demonstrate to us that YOU understand how this business works.

Understanding your process is much more important to us than the headline result.

Submission

Zip up `samples/`, `jobs/`, `archive/`, and `report/` into a single zip and upload it through your personal take-home submission link — the upload page linked in the assignment email you received from Abundant (sent via Ashby, our recruiting platform). Your workspace folder is shared with your host machine, so a zip you create inside the dev container appears directly in the folder you extracted — upload it from there in your browser. Note any key assumptions you made at the top of your report. Questions, or lost your link? Email `careers@abundant.ai`.

Next step: Our research leads will review your report and get back to you with next steps.

Other resources

Hints for finding headroom

You're not graded on which sub-domain you pick — you're graded on whether you find a real failure mode and design a sample that isolates it. Some angles that consistently surface signal:

- **Mine the literature.** Read recent papers, blog posts, and lab releases in the space (last ~6 months). Try recreating tasks from their evals — frontier benchmarks are a cheat-code for finding failure modes that haven't been solved yet, and SOTA plateaus tell you where to dig. As a quick starting point: SWE-Marathon (swemarathon.org) — 20 multi-hour software engineering tasks with tight verification and closed-internet execution.
- **Push on real-world complexity.** Real software engineering isn't a clean greenfield repo. Try environments with sprawling legacy codebases, tangled dependencies, flaky tests, or under-specified requirements. Try outputs that have to fill structured schemas, edit existing artifacts in place, or stay coherent across hundreds of steps.

- **Stretch the horizon until it breaks.** Take a task family models can already finish in a few dozen steps and scale it up — more files, more dependent stages, state carried across the whole run — until success collapses. Tasks just past that cliff are the headroom; chains where an early mistake compounds silently are especially productive.

Step by step

The fastest way to get oriented with Harbor is to run the example task in your workspace in three configurations, then mirror the same flow on your own tasks. Instructions on setting up your environment are located in the [README](#).

1. **Skim the example task's four key files:** `instruction.md`, `task.toml`, `environment/Dockerfile`, `tests/test.sh` (plus the optional `solution/solve.sh`). Note the `[environment.env]` block in `task.toml` — it routes model calls through our gateway, so keep it in every task you author.
2. **Run it with the Oracle agent.** Oracle executes `solution/solve.sh` against the environment — a sanity-check that the task is actually solvable.

```
harbor run -p <example-task> -a oracle
```

Expected reward: 1. If not, the solution or verifier is broken.

3. **Run it with the Nop agent.** Nop does nothing — it just spins up the environment and exits. This proves the verifier doesn't pass on inaction.

```
harbor run -p <example-task> -a nop
```

Expected reward: 0. If you see 1, your verifier is already satisfied by the default state and the task isn't measuring anything.

4. **Run it with the model under test.** This is the configuration you'll capture into `jobs/` for your submission. Terminals inside the dev container automatically carry your gateway credentials into the run.

```
harbor run -p <example-task> -a mini-swe-agent -m anthropic/claude-sonnet-5
```

5. **IMPORTANT: Inspect the agent trajectory.** Inspect `jobs/<job>/<trial>/agent/trajectory.json` and `verifier/reward.txt` to see the real model behavior.

Quality-check each task with `harbor check`

Before submitting, run each of your tasks through `harbor check` to catch common authoring problems (under-specified instructions, gameable verifiers, anti-cheat holes, etc.):

```
harbor check <task-path>
```

Trajectories are produced automatically. When you `harbor run -p <task-path> -a mini-swe-agent -m anthropic/claude-sonnet-5`, Harbor emits a `jobs/<job>/<trial>/` directory containing the ATIF-formatted `agent/trajectory.json`, the verifier output under `verifier/`, and a top-level `result.json`. You don't write trajectories by hand — you submit the captured output. Browse any run with `harbor view jobs`.

Trajectory inspection. Check the agent trajectory to verify that the failure is from genuine task difficulty (e.g. reasoning error, tool call failure, misunderstanding the instruction) and not task design issues (e.g. cheating, missing instruction, verifier design issue).

Other practicals

- **Anthropic API.** `claude-sonnet-5` is the model under test. Your credentials come pre-filled in the workspace `.env` — inside the dev container, Harbor runs, `curl`, and the Anthropic SDK all work with zero setup. If you ever need them outside the container (or `.env` goes missing), your personal `ANTHROPIC_BASE_URL` and `ANTHROPIC_AUTH_TOKEN` are printed below — select the whole block and paste it into your terminal in one go.

```
# Paste into your terminal:
export ANTHROPIC_BASE_URL=https://take-home-automation.vercel.app
export ANTHROPIC_AUTH_TOKEN=thg_SINh0CwL9_E_wwfdMe0yr8iULzjLKZhIU023UJtJwRo
# Keep these credentials private.
```

- **Coding agents.** Use Claude Code or Codex aggressively — this role doesn't reward hand-coding what an agent could write faster. Tell us in the report which loops you automated.
- **Out of scope.** UI, deployment, scaling the verifier infra. Don't pad the report. Don't ship un-edited AI boilerplate.